

Database Team Document

Context Compression Engine — Database Build Guide

Your role: Design and stand up the storage layer that records every compression run so the app has a history view and the team has data to reference for the demo/pitch deck. This is a small, focused scope for a 24-hr hackathon — don't over-engineer it.

1. Your Responsibilities

- Choose and provision the database (Supabase/PostgreSQL recommended — fast to set up, has a JS client, free tier is enough)
- Design the schema (tables below)
- Set up the Supabase project and share credentials securely with Backend team
- Write the SQL to create tables
- (If time allows) Build simple seed/test data so Frontend can develop against realistic data before Backend's pipeline is fully wired up

2. Tech Choice

Supabase (hosted PostgreSQL) — recommended because:

- Free tier, instant provisioning, no server management needed in 24 hrs
- Has a JS client library that Backend can drop straight into API routes
- Built-in table editor UI for quick manual inspection during debugging

3. Step-by-Step Build Plan

Step 1: Provision the Project (15 min)

1. Create a new Supabase project
2. Note down: `SUPABASE_URL` and `SUPABASE_SERVICE_KEY` (service key, not anon key, since

this is a server-side-only app with no auth)

- 3. Share these securely with the Backend team lead (not in a public repo/commit — use an `.env.local` file that’s gitignored)

Step 2: Schema Design (30 min)

Table: `runs` — one row per full compression + validation cycle

Column	Type	Notes
<code>id</code>	<code>uuid</code> , primary key, default <code>gen_random_uuid()</code>	
<code>created_at</code>	<code>timestampz</code> , default <code>now()</code>	
<code>input_preview</code>	<code>text</code>	first ~200 chars of original input, for display in history list
<code>original_tokens</code>	<code>integer</code>	
<code>stage1_tokens</code>	<code>integer</code>	tokens remaining after structural compression
<code>final_tokens</code>	<code>integer</code>	tokens remaining after semantic compression
<code>compression_ratio</code>	<code>numeric(5,2)</code>	percentage, e.g. 72.30
<code>estimated_cost_saved</code>	<code>numeric(10,6)</code>	in USD
<code>question</code>	<code>text</code>	the test question used (auto- generated or user-provided)
<code>answer_full</code>	<code>text</code>	Claude’s answer using full context
<code>answer_compressed</code>	<code>text</code>	Claude’s answer using compressed context
<code>similarity_score</code>	<code>numeric(5,2)</code>	0–100
<code>latency_full_ms</code>	<code>integer</code>	

latency_compressed_ms	integer	
status	text	'success', 'partial' (e.g. Stage 2 skipped), or 'error'

Table: chunks (optional — only build if time allows, useful for a more detailed demo/debug view)

Column	Type	Notes
id	uuid , primary key	
run_id	uuid , foreign key → runs.id	
chunk_index	integer	order within the original text
original_text	text	
compressed_text	text	null if kept verbatim
relevance_score	integer	1–10, from Stage 2 scoring
was_compressed	boolean	

Step 3: Create Tables (20 min)

Run this SQL in the Supabase SQL editor:

```
create table runs (
  id uuid primary key default gen_random_uuid(),
  created_at timestamptz default now(),
  input_preview text,
  original_tokens integer,
  stage1_tokens integer,
  final_tokens integer,
  compression_ratio numeric(5,2),
  estimated_cost_saved numeric(10,6),
  question text,
  answer_full text,
  answer_compressed text,
  similarity_score numeric(5,2),
  latency_full_ms integer,
  latency_compressed_ms integer,
  status text default 'success'
);
```

```

create table chunks (
  id uuid primary key default gen_random_uuid(),
  run_id uuid references runs(id) on delete cascade,
  chunk_index integer,
  original_text text,
  compressed_text text,
  relevance_score integer,
  was_compressed boolean default false
);

create index idx_runs_created_at on runs (created_at desc);
create index idx_chunks_run_id on chunks (run_id);

```

Step 4: Access Policy (10 min)

- For the hackathon MVP, there's no user auth, so:
 - Disable Row Level Security (RLS) on both tables, OR
 - Add a permissive policy allowing all reads/writes (since access is only via the server-side service key, RLS is not a real risk here)
- Do **not** expose the service key to the frontend — only Backend API routes should hold it (server-side env var)

Step 5: Seed/Test Data (20 min, optional but recommended)

Insert 3–5 fake rows into `runs` with realistic-looking numbers so Frontend can build and screenshot the History Panel and dashboard before Backend's real pipeline is finished:

```

insert into runs (input_preview, original_tokens, stage1_tokens, final_tokens, co
values
('def calculate_total(items): ...', 5200, 3100, 1450, 72.12, 0.0231, 'What does c

```

Step 6: Handoff to Backend (15 min)

1. Share the Supabase URL + service key
2. Confirm the exact column names above match what Backend's `/api/compress`, `/api/validate`, and `/api/history` routes expect — this must match the API contracts in the SRS doc exactly
3. Walk Backend through a manual insert once together to confirm the connection works before they build the actual write logic

4. What “Done” Looks Like

- `runs` table exists and accepts inserts from Backend’s API routes without error
- History query (`select * from runs order by created_at desc limit 20`) returns clean, correctly-typed data
- Frontend can render the History Panel using either seed data or real data interchangeably (same shape)

5. Coordination Notes

- Finalize the schema with Backend team **before** they start Step 7 of their build plan (database logging) — schema changes mid-build waste time
- If `chunks` table ends up being skipped due to time, tell Backend and Frontend early so no one builds against it